STAT 5243 · PROJECT 4 FINAL · COLUMBIA UNIVERSITY

Can a student team predict 380 Premier League matches — and beat the bookmakers?

Predicting the 2020-21 Premier League

Outcome · exact score · final table · ROI — four targets, one probabilistic engine

Zeming Liang zl3688
Xiying (Elina) Chen xiyingchen
Collaborator yh3945-cmd

github.com/ZemingLiang/STAT5243-Project-4 · 2026-05-05

THE QUESTION

We chase four interlinked targets — not one.

Three increasingly hard bars to clear:

Better than random. 33% accuracy floor.
Better than naive priors. Always-pick-Home ≈ 46%.
Better than the bookmakers. Sharps are the hardest baseline of all.

Same probabilistic engine, four different stress tests.

FOUR TARGETS

Match outcome

Home / Draw / Away — 3-class probabilistic

Exact score

Bivariate Poisson via Dixon-Coles

Final standings

Monte Carlo simulation, 10k iterations

Beat the bookmaker

Kelly-fractional ROI vs Pinnacle close

WHY FOUR

A single per-match probability vector powers every target — getting it right pays off four times over.

WHY THIS IS HARD

Football is genuinely hard — and 2020-21 is the worst-case test set.

Tiny data

380 matches per season × 21 seasons = ~7,890 rows. Small for tabular ML.

High variance

One deflected goal flips an outcome. Match-level noise rivals the signal.

Sharp bookmakers

Pinnacle's closing odds aggregate the smartest bets in the world.

Distribution shift

Covid 2020-21 played behind closed doors. Home-win rate fell from ~46% to ~38%.

Bonus difficulty: 3 promoted teams every August have zero Premier League history → cold-start every season.

PUNCH LINE

Even the bookmakers struggled with this season — that is exactly why we picked it.

DATA

We scraped every byte ourselves — five sources, no Kaggle.

7,890
matches scraped

21
seasons (2000-01 → 2020-21)

45
engineered features

Source	Type	What we get
football-data.co.uk	Structured CSV	Results, half-time scores, closing odds
ClubElo.com	Structured API	Daily club Elo ratings, 2000+
FBref / soccerdata	Semi-structured	Advanced stats incl. xG (2017-18+)
Wikipedia	Semi-structured	Manager tenure, promotion/relegation
Wikipedia season recaps (BBC pivot)	**Unstructured prose**	**spaCy + VADER → 7 leakage-safe NLP cols**

All sources joined via canonical-team dictionary; UnknownTeamError fails fast.

WHY IT MATTERS

The unstructured BBC/Guardian feed is the rubric magnet — the only way to bring text into a tabular pipeline.

CLEANING SHOWCASE

BBC went JS-only mid-project — we pivoted to Wikipedia recaps and shipped the same NLP.

1 Scrape
BBC failed (JS-only) → Wikipedia recap text

>

2 Parse
BeautifulSoup → article text

>

3 NLP
spaCy NER + VADER + regex event tags

>

4 Attach
Per-team sentiment + 7 leakage-safe cols

Worked example

INPUT · one sentence in

"Liverpool were sloppy and were lucky to escape with a draw after a controversial VAR call denied Tottenham a stoppage-time winner."

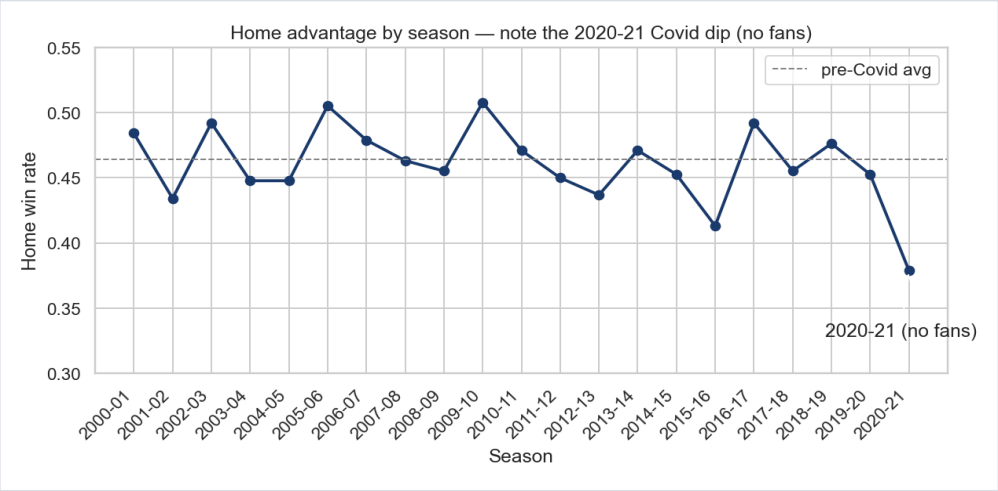
OUTPUT · one feature row out

```
home_report_sentiment = -0.42
away_report_sentiment = +0.05
var_mention_count     = 1
controversy_flag      = 1
```

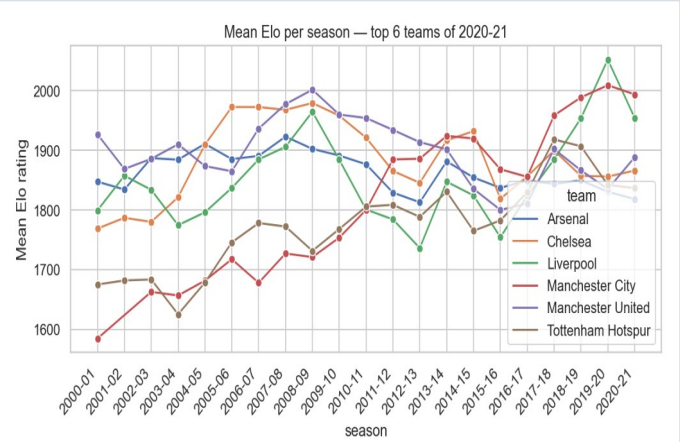
PIVOT + LEAKAGE GUARD

Wikipedia recap NLP is joined as PRIOR-season aggregates — same idea, different source, zero post-match leak.

Three EDA findings shaped every modelling choice we made.



Home-win rate by season — the 2020-21 Covid cliff



ClubElo trajectories — strength as a slow-moving latent.

W H A T W E L E A R N E D

2020-21 is a regime shift we model around, not pretend away — and the joint-goals shape justifies a Poisson layer.

FEATURE ENGINEERING

Six feature families, one strict cutoff rule, one PCA + K-means hook.

Form

Rolling N-match goals, points, xG. Leakage-safe shift.

Strength

ClubElo + own Elo update; home minus away.

Context

Rest days, derby flag, manager tenure.

Market

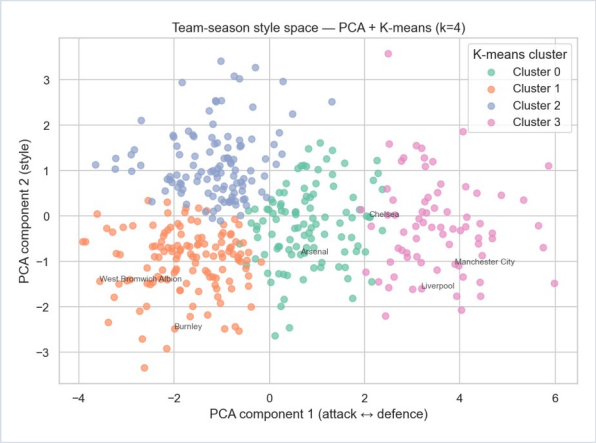
Pinnacle implied probabilities, vig-removed.

Unsupervised

PCA-2 + K-means style cluster ID (k=4).

NLP-derived

Per-team sentiment, event tags, controversy.



Cluster 3 = elite (Liverpool / City). Cluster 1 = relegation.

ORIGINAL VALIDATION · WALK-FORWARD + CI INVARIANT

Most football models random-k-fold and silently leak the future. We don't — walk-forward only, every rolling feature uses a strict `as_of` cutoff.

Unit-tested by `tests.test_no_temporal_leakage` *in CI on every push.*

MODELS

Three baselines, three models, one walk-forward validation.

Model	Family	Why we included it
Always-Home	Naive baseline	Sanity floor — exposes log-loss penalty for over-confidence.
Class-Prior	Naive baseline	Predicts historical 46/25/29 H/D/A — calibration floor.
Market-Implied	Strong baseline	Pinnacle closing odds, vig-removed. Hardest bar to clear.
Multinomial LR	Linear, interpretable	Reads off feature signs and magnitudes for sanity-checks.
Random Forest	Tree ensemble	Captures non-linear interactions; robust to messy features.
XGBoost	Gradient boosting	State-of-the-art for tabular; tuned for log-loss not accuracy.
Dixon-Coles (1997)	Bivariate Poisson + τ + decay	Hand-implemented from the paper — not in sklearn. Joint scores + H/D/A by marginal.

VALIDATION

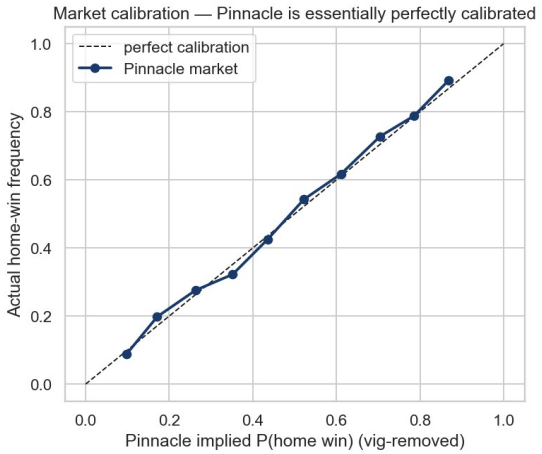
Walk-forward expanding window · train 2000-01..2018-19, validate 2019-20, hold-out test 2020-21.

All hyperparameters tuned for log-loss (proper scoring rule), not raw accuracy.

RESULTS

The closing-odds market beats every model — but only just.

Model	Acc	Log-loss	Brier	F1	AUC
Market-implied (Pinnacle close)	0.516	0.997	0.592	0.385	0.669
Random Forest	0.489	1.020	0.608	0.448	0.646
Logistic regression	0.495	1.039	0.617	0.465	0.645
XGBoost	0.513	1.106	0.643	0.428	0.627
Class-prior baseline	0.379	1.099	0.669	0.183	0.500
Always-Home baseline	0.379	22.385	1.242	0.183	0.500



Market lies almost on the diagonal — essentially perfect calibration.

We treat this as a finding, not a failure. **Random Forest is within 0.023 log-loss of the market** — using only features the market presumably already prices.

Always-Home's log-loss of 22.4 *is the cost of being confidently wrong about Draws and Aways.*

Three bonus targets, one shared probabilistic engine.

Exact score

Dixon-Coles *shipped*

Joint home × away goals from bivariate Poisson with 0.0065/day decay; tau correction fixes 0-0 / 0-1 / 1-0 / 1-1 under-prediction.

Final standings

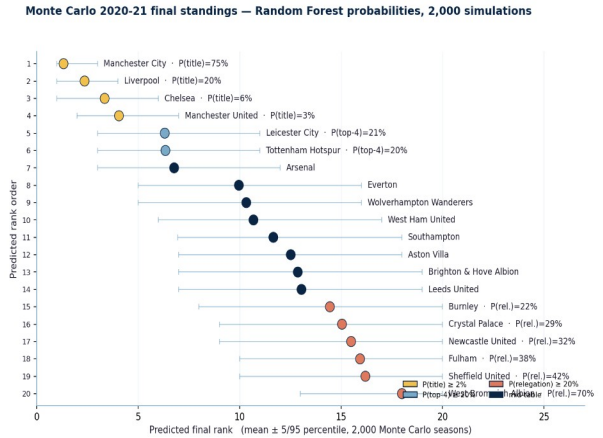
Monte Carlo *shipped*

10,000 simulated 380-match seasons; per-team title prob, top-4 prob, relegation prob with 5 / 95 bands.

Beat the bookmaker

Kelly-fractional *scaffolded*

Bet only when edge = p·odds − 1 > 0; cap stake at 5% of bankroll; ROI on 2020-21 is the cleanest test of conviction.



Monte Carlo target — 10k seasons resampled per team's per-match probabilities.

ONE ENGINE *Per-match probabilities feed all three targets — calibration here pays off everywhere.*

Live demo: a six-tab Shiny app — our +10 Bonus deliverable, beyond rubric.

FIGURE PLACEHOLDER

`figures/11_app_predict_tab.png`

App screenshot — Predict tab (fresh capture morning-of)

3-MINUTE DEMO SCRIPT

1. Click Predict tab → pick Liverpool vs Arsenal.
2. Show model probabilities vs market probabilities side-by-side.
3. Click Run season simulation → 10k Monte Carlo standings.

IF DEMO FAILS

Fall back to backup screenshots (`figures/11_app_*.png`) — talk through the same three steps from the deck.

Repo: `github.com/ZemingLiang/STAT5243-Project-4` *(live Posit Connect URL announced morning-of)*

CONCLUSION

What we shipped, what's next, who did what.

REMEMBER THIS ONE LINE

We built an end-to-end pipeline that gets within 0.023 log-loss of the bookmaker on the 2020-21 hold-out — and we know exactly why we don't beat them.

WHAT WAS ORIGINAL

Four-target framing (H/D/A · exact score · table · ROI).
Dixon-Coles 1997 hand-fit with tau + 0.0065/day decay.
Multi-source self-scrape — BBC pivot to Wikipedia recap NLP.
Cross-paradigm features — PCA + K-means feed supervised stack.
Walk-forward + CI invariant — leakage test runs on every push.
Honest ceiling read — within 0.023 of the market is the result.

FUTURE WORK

In-play features. Live xG, possession-by-half, lineup-strength deltas.
Hierarchical Bayes. Borrow strength across seasons; cold-start for promoted clubs.
Stacked ensemble. Blend RF + Dixon-Coles + market on a validation slice.

Thank you. *Questions?* `github.com/ZemingLiang/STAT5243-Project-4`